

The Python **os** Module

A practical, interview-ready reference covering paths, directories, files, environment variables, and directory trees — with fully commented code examples.

The **os** module is Python's standard interface to the underlying operating system. It lets your code create/remove directories, list folder contents, check and build file paths, read environment variables, and run other OS-level operations — in a way that works across Windows, macOS, and Linux.

1. Finding Out Where You Are

`os.getcwd()` returns the current working directory (the folder your script is running from). `os.chdir()` changes it.

```
import os # Import the standard library os module

# Get the current working directory as an absolute path string
current_dir = os.getcwd()
print(current_dir) # e.g. "/home/youssef/projects/my_script"

# Change the current working directory to another folder
os.chdir("/home/youssef/documents") # Move into this folder
print(os.getcwd())                  # Confirm the change: now prints the new path

# Move back to the original directory
os.chdir(current_dir)
```

2. Creating and Removing Directories

`os.mkdir()` creates a single folder (fails if the parent doesn't exist). `os.makedirs()` creates a full chain of nested folders at once. `os.rmdir()/os.removedirs()` remove empty folders.

```
import os # Needed for directory creation/removal

# mkdir() creates ONE new folder. The parent folder must already exist,
# otherwise a FileNotFoundError is raised.
os.mkdir("reports") # Creates ./reports

# makedirs() creates a full nested path, including any missing parent folders.
os.makedirs("data/2026/september", exist_ok=True)
# exist_ok=True prevents an error if the folders already exist

# rmdir() removes a single EMPTY folder. It fails if the folder has content.
os.rmdir("reports")

# removedirs() removes a folder AND any now-empty parent folders above it.
os.removedirs("data/2026/september")
```

Note: to delete a non-empty folder (with files inside), use `shutil.rmtree()` from the `shutil` module instead — `os.rmdir()` will raise an error if the folder isn't empty.

3. Listing What's Inside a Directory

`os.listdir()` returns a plain list of file/folder names inside a directory. `os.scandir()` is a faster alternative that also gives file-type info without extra system calls.

```
import os # Needed for listdir() and scandir()

# listdir() returns just the NAMES of entries in a folder (not full paths)
entries = os.listdir(".") # "." means the current directory
print(entries) # e.g. ['main.py', 'data', 'README.md']

# scandir() returns DirEntry objects with extra info, and is more efficient
# than calling listdir() + checking each item with os.path.isfile()/isdir()
with os.scandir(".") as it:
    for entry in it:
        if entry.is_file():
            print(f"FILE: {entry.name}")
        elif entry.is_dir():
            print(f"DIRECTORY: {entry.name}")
```

4. os.path: Building and Inspecting Paths

os.path is a sub-module dedicated to path manipulation. It's the safest way to combine folder and file names, since it automatically uses the correct separator ('/' on Linux/macOS, '\\' on Windows).

```
import os # os.path is accessed through the main os module

# join() combines path pieces using the correct separator for the current OS
full_path = os.path.join("data", "2026", "report.json")
print(full_path) # e.g. "data/2026/report.json" (Linux/macOS)
                 # or  "data\\2026\\report.json" (Windows)

# exists() checks whether a path (file or folder) exists on disk
print(os.path.exists(full_path)) # True or False

# isfile() / isdir() check the SPECIFIC type of an existing path
print(os.path.isfile(full_path)) # True if it's a file
print(os.path.isdir("data"))     # True if it's a directory

# split() breaks a path into (directory, filename)
directory, filename = os.path.split(full_path)
print(directory)     # "data/2026"
print(filename)      # "report.json"

# splitext() separates the filename from its extension
name_only, extension = os.path.splitext(filename)
print(name_only)     # "report"
print(extension)     # ".json"

# abspath() converts a relative path into a full absolute path
print(os.path.abspath("report.json"))

# basename() / dirname() get just the filename or just the folder part
print(os.path.basename(full_path)) # "report.json"
print(os.path.dirname(full_path))  # "data/2026"
```

Commonly used os.path functions

Function	Returns
os.path.join(a, b)	Combined path using the correct OS separator
os.path.exists(p)	True if the path exists (file or folder)
os.path.isfile(p)	True if the path is an existing file
os.path.isdir(p)	True if the path is an existing directory
os.path.abspath(p)	The full absolute version of a relative path
os.path.basename(p)	Just the final file/folder name in the path
os.path.dirname(p)	Just the directory portion of the path
os.path.splitext(p)	Tuple of (filename, extension)

<code>os.path.getsize(p)</code>	File size in bytes
---------------------------------	--------------------

5. Renaming and Removing Files

`os.rename()` renames or moves a file/folder. `os.remove()` deletes a single file (not a directory — use `os.rmdir()/shutil.rmtree()` for that).

```
import os # Needed for rename() and remove()

# Create a small file to demonstrate on
with open("draft.txt", "w") as f:
    f.write("Hello!") # Just some sample content

# rename() renames (or moves, if the destination path is in another folder)
os.rename("draft.txt", "final.txt") # draft.txt --> final.txt

# remove() permanently deletes a single file
os.remove("final.txt") # final.txt is now deleted

# Always check a file exists before deleting to avoid a FileNotFoundError
if os.path.exists("final.txt"):
    os.remove("final.txt")
else:
    print("File already removed or never existed.")
```

Caution: `os.remove()` deletes permanently — there is no recycle bin/trash involved. Always double-check the path before calling it.

6. Walking an Entire Directory Tree

`os.walk()` recursively visits every folder inside a starting directory, yielding the current folder path, its sub-folders, and its files at each level. It's the standard way to process a whole folder structure.

```
import os # Needed for walk()

# os.walk(top) walks the directory tree rooted at "top"
# On each iteration it yields a tuple: (current_folder, subfolders, files)
for current_folder, subfolders, files in os.walk("project"):
    print(f"Currently in: {current_folder}")

    for folder_name in subfolders:
        print(f" [DIR] {folder_name}")

    for file_name in files:
        # Build the full path to each file using os.path.join()
        full_path = os.path.join(current_folder, file_name)
        print(f" [FILE] {full_path}")

# Practical example: count every .json file inside a folder tree
json_count = 0
for current_folder, subfolders, files in os.walk("project"):
    for file_name in files:
        if file_name.endswith(".json"):
            json_count += 1

print(f"Total .json files found: {json_count}")
```

7. Reading Environment Variables

`os.environ` behaves like a dictionary of the operating system's environment variables — useful for reading configuration values (API keys, paths, settings) without hard-coding them into your script.

```
import os # Needed to access os.environ

# os.environ acts like a read/write dictionary of environment variables
home_folder = os.environ.get("HOME") # Returns None if the variable isn't set
print(home_folder) # e.g. "/home/youssef" on Linux/macOS

# get() with a default value avoids errors if the variable doesn't exist
api_key = os.environ.get("MY_API_KEY", "no-key-found")
print(api_key)

# Setting an environment variable for the CURRENT process only
# (this does NOT persist after the script ends)
os.environ["DEBUG_MODE"] = "true"
print(os.environ["DEBUG_MODE"]) # "true"
```

Tip: environment variables are commonly used to keep secrets (like API keys) out of source code — set them in your OS or a `.env` file instead.

8. Interview-Ready Summary

Key points that commonly come up when discussing the `os` module:

`os.mkdir()` vs `os.makedirs()`

`mkdir()` creates exactly one folder and fails if parents are missing; `makedirs()` creates the entire nested path at once.

`os.listdir()` vs `os.scandir()`

`listdir()` returns plain name strings; `scandir()` returns richer `DirEntry` objects and is faster when you also need file type info.

`os.remove()` vs `os.rmdir()`

`remove()` deletes a FILE; `rmdir()` deletes an EMPTY directory. Neither can delete a non-empty folder — use `shutil.rmtree()` for that.

`os.path.join()`

Always prefer this over manually concatenating strings with `'/'` or `'\\'`, since it adapts to the current operating system automatically.

`os.walk()`

The standard tool for recursively processing every file and folder inside a directory tree.

`os` module vs `pathlib` module

`os` / `os.path` is the classic, function-based, string-path approach; `pathlib` (newer) offers an object-oriented `Path` class with similar capabilities and a more modern API.

End of guide — every code sample above is fully runnable and commented line-by-line for quick review.